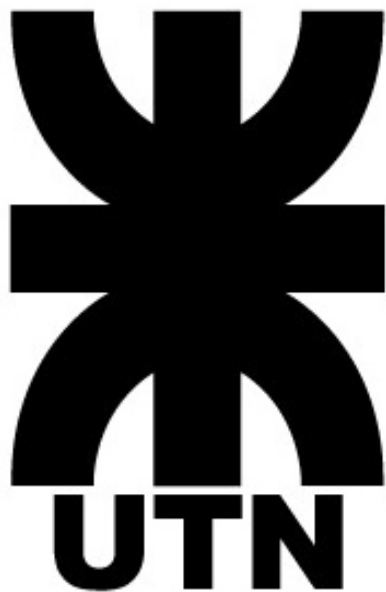




POC OxM

UNIVERSIDAD TECNOLÓGICA NACIONAL FACULTAD REGIONAL ROSARIO
DESARROLLO DE SOFTWARE 2026 COMISIÓN 304 - 06/10/2026

Integrantes del Proyecto

Prisma 	Drizzle 
53742 - Nicolás Bolzico 53952 - Martín Cabrera 52285 - Agustín Gregoret	54326 - Roque Doino 54689 - Juan Pablo Villa

Objetivos de la investigación

El objetivo principal de esta prueba de concepto es analizar, comparar y evaluar dos tecnologías ORM consolidadas dentro del ecosistema moderno de JavaScript y TypeScript: Prisma y Drizzle.

Con el fin de ofrecer un marco de decisión aplicable a proyectos reales de software, la investigación busca comprender no solo el funcionamiento técnico de cada herramienta, sino también contrastar sus filosofías de diseño, facilidad de uso, rendimiento y presencia en el mercado laboral, complementándolo con una demostración empírica resolviendo casos de uso equivalentes.

Introducción a los OrM

Antes de explicar qué es un ORM, es importante entender el problema que estas tecnologías intentan resolver.

En una aplicación moderna, los datos se manipulan utilizando las estructuras propias del lenguaje de programación. En JavaScript o TypeScript, por ejemplo, se trabaja con objetos, clases, propiedades y arreglos para representar la información del sistema. Sin embargo, cuando esa información debe almacenarse de manera persistente, normalmente se recurre a una base de datos relacional cuya forma de organización es tabular (tablas, filas, columnas, claves primarias y foráneas). Esto genera un conflicto estructural conocido como desfase objeto-relacional (Object-Relational Impedance Mismatch).

Como consecuencia, los desarrolladores históricamente debían asumir una enorme sobrecarga de código repetitivo para realizar esta conversión de forma manual: transformar filas en objetos y construir cada sentencia SQL (INSERT, UPDATE, DELETE). Este enfoque no solo ralentizaba el desarrollo, sino que representaba una fuente crítica de errores en tiempo de ejecución, ya que cualquier errata en el nombre de una columna, un desajuste en los tipos de datos o un parámetro mal posicionado pasaba inadvertido en desarrollo y hacía fallar la aplicación recién en producción.

Un ORM (Object-Relational Mapping) es una capa intermedia entre la aplicación y el motor de base de datos que automatiza la correspondencia entre las estructuras del código y los registros de persistencia. Provee una interfaz de alto nivel que sigue la lógica del paradigma orientado a objetos, lo que nos permite abstraernos del motor de base de datos subyacente. De esta forma, el

desarrollador interactúa con la base de datos mediante entidades y métodos nativos del lenguaje.

Es crucial destacar que utilizar un ORM no significa que SQL desaparezca. La base de datos continúa procesando sentencias SQL tradicionales; lo que cambia es que el desarrollador delega la construcción, ejecución, parametrización y mapeo de esas consultas a una interfaz de alto nivel más segura y mantenible.

Evolución e Historia de los ORM

El desarrollo de herramientas para el mapeo objeto-relacional ha atravesado diversas etapas para adaptarse a las necesidades tecnológicas de cada época, evolucionando desde soluciones rudimentarias hasta interfaces modernas enfocadas en seguridad de tipos y rendimiento:

- **Orígenes y Consolidación (1990 - mediados de 2000):** Las primeras implementaciones surgieron con herramientas como TopLink para Smalltalk en la década de 1990. Posteriormente, a principios de los 2000, Hibernate estableció el estándar de persistencia en el ecosistema Java, seguido por Django ORM y SQLAlchemy en Python a mediados de la misma década.
- **Enfoque en Type-Safety y la Era Moderna (2000s tardíos - Presente):** A finales de los 2000, Entity Framework adaptó este paradigma al ecosistema .NET. Con el auge de TypeScript y JavaScript a mediados de la década de 2010, aparecieron soluciones como TypeORM. Finalmente, en los últimos años, ORMs de nueva generación como Prisma y Drizzle han redefinido el estándar al ofrecer seguridad de tipos (Type-Safety) avanzada y mejor integración con arquitecturas modernas.

Adopción de los ORM:

Como referencia sobre el nivel de adopción de estas tecnologías, puede observarse la Python Developers Survey 2024, realizada por la Python Software Foundation y JetBrains. Aunque esta encuesta corresponde al ecosistema Python y no específicamente a JavaScript o TypeScript, resulta útil como indicador general, ya que los ORM no son una tecnología exclusiva de un lenguaje determinado, sino un enfoque utilizado ampliamente en distintos ecosistemas de desarrollo.

Dentro de los más de 30 mil desarrolladores web encuestados, solo un 14% indicó utilizar *Raw SQL*, es decir, hacer las consultas sin ningún ORM.

Estos datos permiten observar que los ORM tienen una presencia muy importante dentro del desarrollo de aplicaciones actuales. Su adopción en distintos lenguajes y ecosistemas demuestra que se han convertido en una tecnología fundamental para simplificar el acceso a bases de datos, reducir trabajo repetitivo y mejorar la mantenibilidad del código.

Organización y Metodología del Documento

Para llevar a cabo una evaluación exhaustiva, objetiva y reproducible de las tecnologías seleccionadas, el presente informe estructura su análisis y desarrollo en seis bloques fundamentales:

1. Presentación de cada tecnología: Introduce a Prisma y Drizzle ORM detallando el año de surgimiento de cada una y el contexto de la industria en el que aparecieron.
2. Presentación de cada tecnología: Introduce a Prisma y Drizzle detallando el año de surgimiento de cada una y el contexto de la industria en el que aparecieron.
3. Factores Técnicos (Arquitectura y Rendimiento): Examina el mecanismo interno de comunicación con el motor de base de datos, el peso final del paquete (bundle size) y la compatibilidad con infraestructuras modernas de baja latencia (Serverless y Edge computing). Asimismo, profundiza en la flexibilidad de sus interfaces de consulta y la solidez de la seguridad de tipos (Type Safety) en tiempo de desarrollo.
4. Mercado Laboral y Empleabilidad: Analiza la inserción de cada ORM en la industria tecnológica contemporánea. Se identifica el perfil de las organizaciones que demandan cada tecnología y se evalúa si su conocimiento constituye un estándar de mercado prioritario o un valor agregado diferencial en el perfil profesional.
5. Implementación y Demostración Práctica (PoC): Comprende el diseño y la implementación de un caso de uso común sobre un modelo de datos relacional para resolver de forma equivalente las operaciones cotidianas de un sistema real.
6. Conclusiones y Veredicto Final: Presenta un balance crítico de ventajas, desventajas identificados en cada tecnología, proporcionando una matriz de decisión orientada a determinar qué herramienta resulta más conveniente adoptar según las prioridades de rendimiento, arquitectura y equipo de un proyecto determinado.

A través de esta metodología integral, la investigación busca proporcionar un criterio fundamentado y aplicable para la toma de decisiones técnicas en entornos de desarrollo reales.

1. Presentación

Prisma

Prisma fue fundado originalmente en 2016 enfocado en la construcción de APIs, para luego reenfocarse por completo como un ORM en el año 2020 con el lanzamiento de Prisma 2.0 y alcanzar su versión estable en 2021. Se trata de un proyecto de código abierto impulsado por una empresa privada respaldada por capital de riesgo (VC-backed), la cual ha desarrollado a su alrededor un ecosistema comercial en la nube que incluye soluciones como Prismaverse, Prisma Postgres y Prisma Accelerate.

Drizzle

Drizzle surgió en el año 2022 y experimentó un fuerte crecimiento durante el período 2023-2024 como una respuesta directa a las abstracciones pesadas sobre SQL, promoviendo la filosofía de que quien conoce SQL ya conoce Drizzle. Nació como un proyecto de desarrollo abierto independiente bajo la licencia Apache-2.0, hasta su adquisición por parte de la empresa PlanetScale en marzo de 2026, manteniendo su modelo de desarrollo Open Source.

2. Factores No Técnicos

Prisma

En cuanto a la salud de su ecosistema y popularidad, Prisma cuenta con más de 47.600 estrellas en GitHub, una comunidad oficial en Discord de aproximadamente 10.300 miembros y una masiva adopción en la plataforma npm, alcanzando alrededor de 55,3 millones de descargas mensuales para julio de 2026. Su proceso de mantenimiento y resolución de errores está formalmente estructurado a través de GitHub Issues mediante plantillas de reproducción mínima, siguiendo un ciclo que abarca desde el triaje y el diagnóstico de sus componentes (CLI, runtime y adaptadores) hasta la corrección, pruebas de regresión y notas de lanzamiento.

La curva de aprendizaje y el proceso de incorporación resultan de nivel bajo a medio, siendo una opción sumamente amigable para desarrolladores orientados al frontend o full-stack gracias a la simplicidad de su lenguaje declarativo de modelado. Asimismo, ofrece una excelente integración con herramientas de

inteligencia artificial debido al masivo volumen de datos públicos disponible y a su arquitectura Agent-Native (v8) enfocada en salidas estructuradas.

Drizzle

Drizzle presenta un ecosistema en constante expansión, respaldado por más de 36.000 estrellas en GitHub, más de 1.400 forks, un servidor de Discord con aproximadamente 12.000 integrantes y un volumen en npm que alcanzó los 76 millones de descargas mensuales hacia septiembre de 2026. Su esquema de mantenimiento destaca por un desarrollo completamente abierto mediante solicitudes de extracción, reporte de fallos en GitHub e interacción fluida dentro de su comunidad.

Respecto al onboarding, la herramienta no presenta curva de aprendizaje para aquellos desarrolladores que ya poseen bases sólidas en SQL, JavaScript y TypeScript. Sin embargo, la exigencia de aprendizaje puede incrementarse si se desconocen los conceptos fundamentales del modelo relacional y el lenguaje de consultas de bases de datos.

3. Factores Técnicos

Prisma

En materia de arquitectura e infraestructura, el motor de Prisma fue reescrito completamente a TypeScript en sus versiones más recientes, lo que permitió reducir el tamaño final del paquete compilado a un máximo de 1,6 MB con un cliente ligero de alrededor de 100 KB, alcanzando aproximadamente un 90% del rendimiento que ofrece un controlador nativo como pg. Por su parte, la definición del esquema y la seguridad de tipos se gestionan mediante un modelado declarativo en un archivo centralizado, lo que facilita la generación automática de tipos estáticos y de un cliente a medida para prevenir errores durante la etapa de compilación. En cuanto al soporte de motores de almacenamiento, abarca bases de datos relacionales como PostgreSQL, MySQL, MariaDB, SQLite y CockroachDB, además de alternativas NoSQL como MongoDB.

Drizzle

La arquitectura de Drizzle está diseñada como una capa liviana construida directamente sobre los controladores nativos de la base de datos (como node-postgres o mysql2), prescindiendo de un motor de ejecución intermedio.

Esto le permite ser una solución ultraligera con un peso aproximado de 7,4 KB comprimido, óptima para entornos Serverless y Edge computing al mitigar los problemas de inicio en frío (cold starts). Sus interfaces de consulta combinan una API de estilo SQL junto con una API relacional, permitiendo incorporar fragmentos de SQL puro sin perder la comprobación de tipos en TypeScript. El ecosistema se complementa con Drizzle Kit para la gestión de migraciones y Drizzle Studio como interfaz gráfica, enfocándose exclusivamente en motores relacionales tales como PostgreSQL, MySQL, SQLite, MSSQL y CockroachDB.

4. Mercado Laboral y Empleabilidad

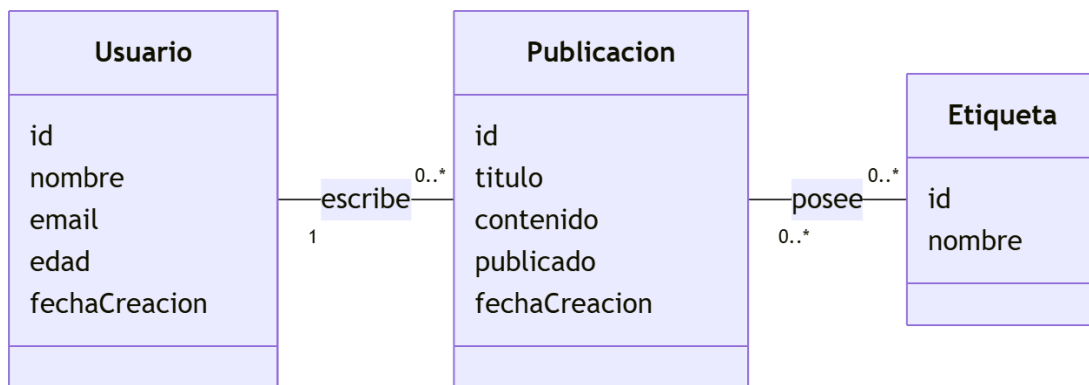
Prisma

Actualmente, Prisma se posiciona como una de las herramientas más solicitadas dentro del ecosistema Node.js y TypeScript, consolidándose como un estándar de facto tanto en startups y productos mínimos viables como en empresas de mayor escala que aprovechan sus extensiones de aceleración para arquitecturas serverless. Su alta adopción se ve reforzada por una integración nativa y directa con los principales marcos de desarrollo web modernos, tales como Next.js, NestJS y Remix.

Drizzle

Por otro lado, Drizzle registra una demanda en rápido crecimiento, especialmente en startups tecnológicas y proyectos orientados a Next.js o infraestructuras sin servidor. Si bien en la actualidad no suele figurar como un requisito indispensable o excluyente en la mayoría de las búsquedas laborales del sector, el dominio de esta tecnología representa un valor diferencial muy apreciado para desarrollos que priorizan la optimización y el máximo rendimiento.

5. Implementación y Demostración Práctica (PoC)



El dominio elegido corresponde a un caso de uso real de un sistema de publicaciones. Este modelo de datos contempla una relación uno a muchos (1:N) entre **Usuario** y **Publicación** (un usuario escribe múltiples publicaciones), así como una relación muchos a muchos (N:M) entre **Publicación** y **Etiqueta** (una publicación puede poseer varias etiquetas y una etiqueta asociarse a múltiples publicaciones).

Para evaluar cuantitativa y cualitativamente a Prisma y Drizzle, se implementarán y analizarán las siguientes tres operaciones fundamentales en cada tecnología:

1. **Modelado / Definición de Esquema:** Creación y estructuración de las entidades y sus relaciones.
2. **Escritura / Registro de Datos:** Alta de un nuevo usuario en el sistema.
3. **Lectura / Búsqueda por Criterio:** Consulta de publicaciones filtradas por la etiqueta "meme" cuyo autor sea mayor de 18 años.

Código en <https://github.com/TinchoLC/PoC-OxM---UTN-FRRO-DSW-2026>

Prisma

- En términos de sintaxis y ergonomía de desarrollo, Prisma destaca por un enfoque de consulta altamente declarativo basado en la construcción de objetos anidados mediante propiedades como `include`, `select` y `where`, simplificando la manipulación de relaciones complejas sin necesidad de redactar sentencias SQL explícitas.

Drizzle

- Drizzle, por su parte, ofrece mayor flexibilidad al disponer de dos modalidades de consulta: un estilo fluido cercano a la estructura estándar de SQL empleando métodos encadenados, o bien una API relacional diseñada para explorar entidades vinculadas de forma más intuitiva.

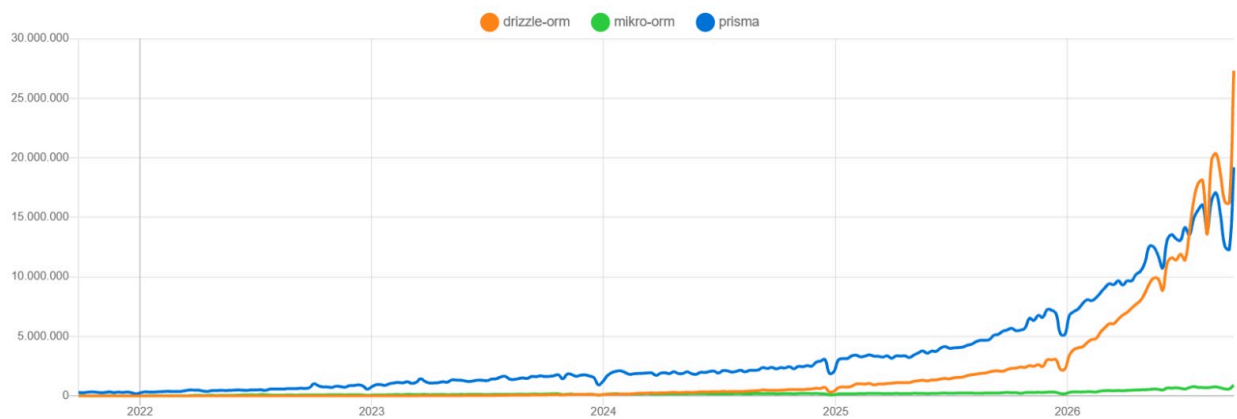
6. Conclusiones

Ecosistema y Tendencia de Mercado (El punto de inflexión):

Durante el desarrollo de esta investigación se evidenció un cambio histórico en la comunidad: mientras que Prisma había sido desde sus inicios el líder indiscutido y dominante del ecosistema con una ventaja abrumadora, Drizzle experimentó un crecimiento tan acelerado en este último período que logró superarlo por primera vez en volumen de descargas mensuales en npm (~76 M frente a ~55,3 M). Esto quiebra la hegemonía histórica de Prisma y consolida una fuerte inercia a favor de Drizzle de cara al futuro del desarrollo en TypeScript. Sin embargo, es clave recalcar que Prisma al tener más antigüedad tiene proyectos legacy, cosa que Drizzle por ser más nuevo no tiene.

También podemos evidenciar con una visión aproximada de los últimos cinco años de descargas semanales de cada tecnología en npm, la caída de descargas de ambas son de la misma proporción.

Downloads in past 5 Years ▾



Es importante señalar que el impulso reciente de Drizzle responde al entusiasmo de la comunidad por el lanzamiento de su versión 1.0. Sin embargo, el panorama continúa abierto: Prisma reaccionó modernizando su infraestructura en Prisma 7 y v8 para resolver sus principales puntos débiles frente a Drizzle —el peso del bundle y el rendimiento—, consolidando una competencia directa por el estándar de desarrollo en TypeScript.

Proyectos Cortos (MVPs, Hackathons, Prototipos) → Prisma:

Ofrece una experiencia batteries-included con fricción casi nula. Su esquema declarativo (schema.prisma), el cliente autogenerado y herramientas como Prisma Studio permiten tener la persistencia operativa y tipada en minutos, sin necesidad de configurar drivers manualmente ni diseñar sentencias SQL. Prioriza al 100% el Time-to-Market.

Proyectos Largos (Sistemas Core, Enterprise, Alta Escala) → Drizzle:

A medida que el sistema madura, la optimización de consultas se vuelve crítica. Drizzle evita la deuda técnica al operar en TypeScript puro sin capas intermedias opacas, modularizar esquemas con facilidad y otorgar control total sobre el SQL ejecutado. Esto facilita auditorías por parte de DBAs y previene cuellos de botella cuando el volumen de datos crece drásticamente.

Matriz de Decisión por Infraestructura y Arquitectura:

Prisma es superior cuando: El equipo es de perfil Frontend/Full Stack, se requiere soporte políglota (bases relacionales + NoSQL como MongoDB).

Drizzle es superior cuando: Se despliega en Serverless o Edge computing (paquete de ~7,4 KB frente a ~1,6 MB), o cuando la lógica de negocio exige consultas SQL avanzadas, reportes y optimización fina de índices.

Mercado Laboral:

En la práctica laboral contemporánea, el dominio de un ORM específico rara vez constituye un requisito excluyente en las búsquedas de la industria. Las organizaciones priorizan perfiles con fundamentos sólidos en TypeScript, patrones de arquitectura backend y diseño de bases de datos relacionales; una vez contratado el desarrollador, las empresas asumen que la adaptación a la sintaxis y particularidades del ORM adoptado por el proyecto se resuelve de manera natural durante el onboarding.

7. Bibliografía

General:

<https://npmrends.com/>
<https://wiki.c2.com/?ObjectRelationalImpedanceMismatch>
<https://www.oracle.com/technical-resources/articles/enterprise-architecture/toplink-history.html>
<https://hibernate.org/orm/documentation/>
<https://maetl.net/the-rise-and-fall-of-object-relational-mapping>
<https://lp.jetbrains.com/python-developers-survey-2024>
<https://survey.stackoverflow.co/>

Drizzle:

<https://github.com/drizzle-team/drizzle-orm>
<https://github.com/drizzle-team/drizzle-orm/discussions/>
<https://orm.drizzle.team/docs/overview>
<https://newreleases.io/project/github/drizzle-team/drizzle-orm/release/0.26.0>
<https://discord.com/servers/drizzle-team-1043890932593987624>
<https://docs.prisma.io/docs/orm/more/comparisons/prisma-and-drizzle>
<https://orm.drizzle.team/>
<https://planetscale.com/blog/drizzle-joins-planetscale>

Prisma:

<https://www.prisma.io/blog/how-prisma-orm-became-the-most-downloaded-orm-for-node-js>
<https://www.prisma.io/docs/orm/v7/core-concepts/data-modeling>
<https://www.pingcap.com/article/understanding-prisma-orm/>
<https://keepcoding.io/blog/prisma-orm/>
<https://www.prisma.io/docs/orm/v7/core-concepts/supported-databases/sql-server>
<https://laboral.perceptual.cl/empleos/desarrollador-backend-node-js-typescript-contrato-indefinido-hibrido>
<https://www.dice.com/job-detail/1627c7d0-b847-4dce-b661-6b607abc297a>
<https://bebee.com/mx/jobs/senior-back-end-developer-engineering-bsc-goods-miguel-hidalgo-ciudad-de-mexico--jooble--4281809054886526335--6d762e71df9325fd706588acd8a7e217>